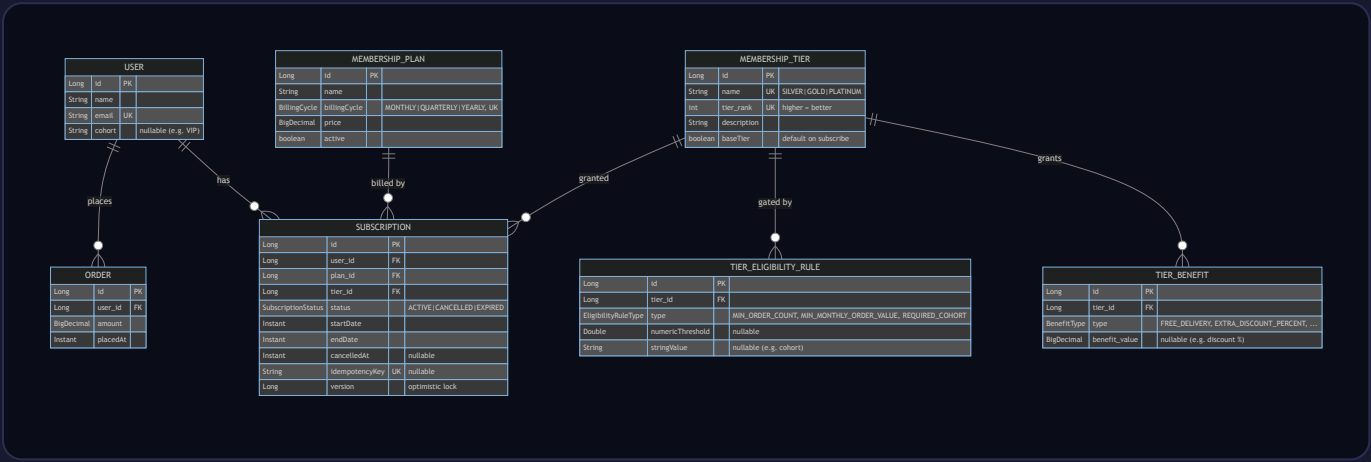
🗃️ Entity Design — FirstClub Membership Program

Normalised, data-driven model. Tiers, benefits and promotion rules are rows, not code — fully configurable.

↗ Demo console ↗ Architecture & flow ER diagram Entity reference Design rationale

1 · Entity-relationship diagram

A subscription binds a **user** to a **plan** and a **tier**. A tier owns many configurable **benefits** and **eligibility rules**. **Orders** feed tier evaluation.



2 · Entity reference

Entity	Responsibility	Key design points
User	Platform member	`cohort` drives cohort-based eligibility; email unique.
MembershipPlan	Billing cadence + price	`BillingCycle` enum carries duration months → expiry math. One plan per cycle.
MembershipTier	A tier & its perks/gates	`rank` orders tiers for upgrade/downgrade; `baseTier` = default. Owns benefits + rules (cascade).
TierBenefit	One configurable perk	Stored as rows (not flags) → add/remove/retune perks at runtime.
TierEligibilityRule	One promotion criterion	Maps to a Strategy by `type`; ALL rules of a tier must pass.
Subscription	User↔Plan↔Tier for a window	`@Version` optimistic lock; unique `idempotencyKey`; lazy expiry.
Order	A purchase	Raw signal for order-count & monthly-spend rules.

3 · Design rationale (abstractions, extensibility, modularity)

Decision	Why
Benefits & rules as **child rows**, not boolean columns	New perks/criteria need no schema or code change — the program is configured as data and editable via APIs.
Plan and Tier are **separate** entities	Orthogonal concerns: billing cadence vs. benefit level. A user keeps their plan while moving tiers.
Enums for cycle / benefit / rule / status	Type-safety + each enum constant pairs with a strategy or duration, keeping behaviour close to data.
`tier_rank` as an integer	Single source of truth for ordering → trivial, correct upgrade/downgrade comparisons.
Version + idempotency on Subscription	Concurrency-safe lifecycle without pessimistic DB locks (see Architecture §5).
Reserved words mapped (`tier_rank`, `benefit_value`)	Portable across H2 / other SQL engines.